```
urlpatterns = [
    ...
    path('bands/', include("bands.urls")),
]
```

Register all the routes in bands.urls.py under the bands/ prefix.

When adding more views to the `bands` app, you now only need to register the URL in RiffMates/bands/urls.py. Any new URL routes there are included under `bands/` through the above change to the main route file.

To complete your view, you'll need a template. Recall from listing 4.2 that the `render()` shortcut passed in `musician.html` as the template to be rendered. Feel free to be more creative, listing 4.7 contains a bare-minimum template. Create a new file named RiffMates/templates/musician.html for this content.

Listing 4.7 Template for musician detail information

```
<!-- RiffMates/templates/musician.html -->
{% extends "base.html" %}

{% block title %}
  {{block.super}}: Musician Details
{% endblock %}

{% block content %}
  <h1>{{musician.first_name}} {{musician.last_name}}</h1>

  <p> Was born {{musician.birth}}. </p>
{% endblock content %}
```

Your new view is ready to go. Fire up the old development server and visit the web page at http://localhost:8000/bands/musician/1/ to see Steve Vai's musician Details page. It will look something like 4.6, depending on how seriously you took the challenge to be creative.

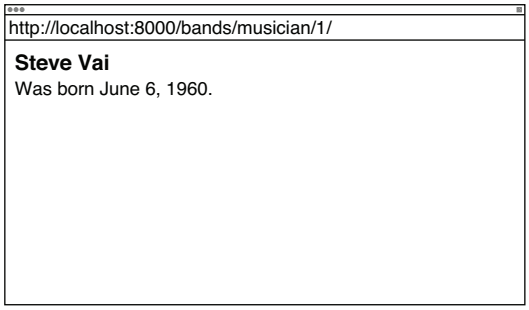


Figure 4.6 Musician view showing Steve Vai

By editing the URL and changing the numeric value, you can query different musicians in the database. Shortly, you'll be creating a listing page that has links to many musicians, but you want to make sure the Details pages work before you get there. Test a few different `Musician` ID values.

You should also check an ID value that doesn't exist, first to ensure the error handling code is working and second so you can experience the wonder that is a Django Error page. When your code raises an `Http404` exception, Django does one of two things, depending on the value of `DEBUG` in your configuration RiffMates/RiffMates/settings.py file. In a production environment, where `DEBUG=False`, Django shows a